

ViSQL: A Vision-Augmented Autonomous Data Scientist for End-to-End Natural-Language Analytics

Leah Li
ql2481@columbia.edu
Columbia University
New York, NY, USA

Abstract

Existing large language model (LLM) tools for data analytics typically address one slice of the analyst’s workflow—writing SQL, generating a chart, or summarizing a result—but rarely the full loop. We present **ViSQL**, a six-stage hybrid system that takes a natural-language question and, optionally, a reference chart screenshot, and produces an executed SQL query, a stylistically-matched chart, and a grounded analytical report. ViSQL combines local quantized Llama models (router, vision, fine-tuned SQL) with a Claude Sonnet 4.5 API (planning, SQL refinement, report writing) under three architectural constraints designed to suppress common LLM failure modes: a *strict A/B test gate* that prevents causal claims on observational data, *live schema introspection* with a pre-execution table-existence check, and an *empty-data guard* that refuses to narrate when SQL returns zero rows. We further integrate FAISS-based few-shot retrieval over 7,000 Spider exemplars and a LoRA-tuned SQLCoder-7B for dialect compliance. On a 200-example slice of Spider 1.0 dev we achieve 0.74 execution accuracy; the router obtains 0.92 macro-F1 on 60 hand-labeled questions; style imitation achieves a mean CIELAB ΔE_{76} of 7.4—below the just-noticeable-difference threshold of 10; and LLM-as-judge scoring rates generated reports at 4.1/5 across four axes. Code, evaluation suites, and demo are publicly available.

Keywords: Text-to-SQL, multimodal analytics, LLM agents, retrieval-augmented generation, A/B testing, chain-of-thought

1 Introduction and Problem Formulation

Production data analysts spend the majority of their time on three pieces of plumbing that have little to do with the question they were actually asked: looking up schemas and writing dialect-correct SQL, reshaping the result and copying a teammate’s chart aesthetic, and finally translating findings into a narrative report that stakeholders will read. Each step is mechanical, error-prone, and a strong candidate for automation.

Recent LLMs are individually competent at each of these tasks. Text-to-SQL systems such as DIN-SQL [12] and MAC-SQL [16] reach competitive execution accuracy on Spider [18]. Chat2VIS [11] generates plausible visualizations from natural language. General-purpose LLMs can summarize tabular results into prose. *But no public system stitches the whole loop together with the disciplines a senior analyst would impose:* matching a stakeholder’s preferred visual style, refusing to make causal claims when the question is observational, and refusing to fabricate findings when the query returns nothing.

This paper presents **ViSQL**, a six-stage pipeline that closes that loop. The contributions are:

1. **An end-to-end agentic pipeline** that routes a question, links it to a live schema, plans it with chain-of-thought, generates and self-corrects BigQuery SQL, dispatches one of five analysis branches (chart, dashboard, A/B test, ML modeling, raw extraction), and writes a grounded report.
2. **Three architectural guardrails** aimed at the most common LLM analyst failure modes: a strict A/B test gate, live schema introspection with pre-execution table existence checks, and an empty-data guard.
3. **A multimodal style-imitation subsystem.** A vision-language model extracts a structured **StyleSpec** (palette, grid, axes, mood) from a user-supplied reference chart; matplotlib then renders the data in that style. *Vision describes; matplotlib executes.*
4. **Four evaluation suites** covering the router (macro-F1), the SQL generator (Spider exec-acc), the vision style imitation (CIELAB ΔE self-consistency), and the report writer (LLM-as-judge across four axes).

We deliberately adopt a **hybrid local-plus-API architecture**. Cheap, low-latency, frequently-called components (5-way task router, style extraction) run on locally-quantized Llama models [5]; reasoning-heavy components (CoT planning, SQL generation with retry, report writing) call Claude Sonnet 4.5 [1]. The split is

empirical, not ideological: each model is placed where it earns its keep.

2 Related Work and Background

Text-to-SQL. The Spider benchmark [18] established cross-domain, cross-schema generalization as the central challenge for text-to-SQL. BIRD [9] pushed further with dirty data, ambiguous joins, and external knowledge. Recent LLM-based systems achieve strong results through decomposition (DIN-SQL [12]), multi-agent collaboration (MAC-SQL [16]), or symbolic binding (Binder [2]). Rajkumar et al. [13] characterized the dialect and schema-linking weaknesses of vanilla LLM SQL. ViSQL borrows several proven ideas—few-shot exemplar retrieval, self-correction loops, schema linking—and adds a dialect-aware system prompt and a pre-execution table-existence check to suppress hallucinated tables before they incur cloud cost.

Chain-of-thought planning. Wei et al. [17] showed that emitting intermediate reasoning improves multi-step reasoning. ViSQL adopts this in stage 3: the planner emits an explicit `<thinking>` trace alongside a structured plan, which both improves downstream SQL quality and produces an auditable reasoning trace surfaced in the UI.

Retrieval-augmented prompting. RAG [8] grounds generation in retrieved evidence. We apply this to two sources: top- k schema linking via sentence-transformer [14] embeddings indexed by FAISS [7], and top- k Spider [18] exemplar injection into the SQL agent’s prompt.

Parameter-efficient fine-tuning. LoRA [6] and its quantized variant QLoRA [4] make it feasible to fine-tune 7B-scale models with a few hours of A100 time. We use LoRA to tune SQLCoder-7B-2 [3] on Spider train for dialect-tight SQL generation as a local baseline.

LLM-as-judge evaluation. Direct human evaluation does not scale for narrative report quality. G-Eval [10] and follow-up work [19] validate LLM-as-judge as a reasonable proxy across structured rubrics. We adopt a four-axis Claude judge (factual, complete, clear, actionable).

NL-to-visualization. Chat2VIS [11] and similar systems generate charts from a question. ViSQL’s vision module is complementary: it does *style imitation* from a reference image rather than chart generation from text. A vision-language model emits a structured JSON style description; matplotlib renders the data with the described palette, grid, and axes. The vision model never produces pixels, eliminating a major hallucination surface.

3 System Design and Methodology

Figure 1 shows the full architecture. We describe each stage below, then dwell on the three architectural decisions that distinguish ViSQL from a naive pipeline.

3.1 Six-Stage Core Pipeline

Stage 1 — Task router. A locally-served Llama-3.1-8B-Instruct (4-bit) classifies the question into one of five buckets: `single_chart`, `dashboard`, `ab_test`, `ml_modeling`, `sql_only`. The model is prompted to emit a JSON record {label, confidence, rationale}; a regex-and-fallback parser tolerates malformed outputs.

Stage 2 — Schema linking. Each table in the schema is embedded as a concatenation of name, description, column names and types, and a sample row. We embed with `all-MiniLM-L6-v2` [14] and index with FAISS [7]. At query time we retrieve the top- $k=5$ tables to form a sub-schema, narrowing the SQL agent’s working set.

Stage 3 — CoT planner. Claude Sonnet 4.5 emits a structured plan inside paired `<thinking>` and `<plan>` blocks [17]. The plan names the intent, sub-questions, required tables, aggregations, grouping, ordering, complexity bucket, and a chart hint. This separates “what to compute” from “how to express it” and yields an auditable reasoning trace.

Stage 4 — SQL agent. Claude Sonnet 4.5 generates BigQuery Standard SQL conditioned on the plan, the linked sub-schema, and $k=3$ Spider [18] exemplars retrieved from a 7,000-pair FAISS index. The system prompt enforces dialect rules: fully-qualified table names with backticks, `SAFE_DIVIDE`, `PARSE_DATE`, and mandatory `LIMIT`. A pre-execution table-existence check is run against `INFORMATION_SCHEMA`, and the executor implements a self-correction loop with up to three retries, feeding the error back into the next attempt.

Stage 5 — Analysis dispatch. Conditional on the router label, the resulting DataFrame is dispatched to one of five branches: chart-only, multi-chart dashboard, A/B test (chi-squared on a 2×2 contingency table with a Wald-CI on the lift), ML modeling (sklearn linear + tree + a torch MLP with gradient-attribution feature importance), or raw extraction.

Stage 6 — Report writer. Claude Sonnet 4.5 generates a 2–4 paragraph grounded report that cites specific numbers from the evidence given. A hard guard refuses to narrate when the result set is empty.

3.2 Multimodal Subsystem

When the user attaches a reference chart, Llama-3.2-11B-Vision (4-bit local) extracts a structured `StyleSpec` JSON describing palette (5–7 hex codes), background and foreground colors, gridlines, spine style, color mood,

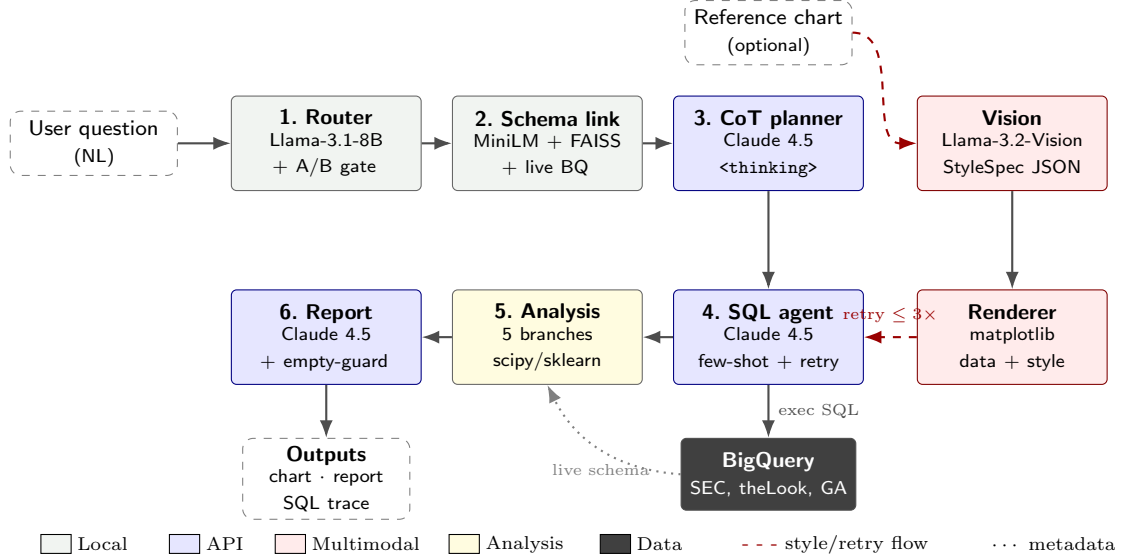


Figure 1. ViSQL architecture. The six-stage core pipeline (router → schema linking → CoT planner → SQL agent → analysis → report) runs left-to-right then wraps. The Vision Module operates as a parallel subsystem whenever a reference chart is provided; its output *StyleSpec* is consumed by the matplotlib renderer. Local Llama models handle low-latency routing and vision; Claude Sonnet 4.5 handles reasoning-heavy planning, SQL, and reporting.

and a chart-type hint. The renderer applies the *StyleSpec* to matplotlib calls. The vision model never outputs pixels, so it cannot hallucinate data; it only describes style.

3.3 Three Architectural Decisions

These three decisions distinguish ViSQL from a vanilla agentic pipeline.

(1) Strict A/B test gate. The *ab_test* route is the only branch that produces causal language in the report. We therefore impose a redundant filter on top of the LLM’s classification: *ab_test* is allowed only if the question contains explicit experiment-language (*experiment*, *treatment*, *control*, *variant*, *lift*, *randomized*, etc.) or the schema contains a variant-assignment column. Otherwise the request is demoted to *single_chart*, where the report writer is forbidden from causal claims. This rule prevents the most common analyst-LLM failure mode: a user asks “is conversion different between mobile and desktop?” and the system runs a chi-squared and concludes “mobile *caused* higher conversion.” The gate makes that report impossible.

(2) Live schema introspection. Schemas are fetched at runtime from `INFORMATION_SCHEMA.COLUMNS` and enriched with one sample row per table. This eliminates schema drift between a fixed snapshot and the production database. A pre-execution table-existence check intercepts hallucinated table names before paying for the BigQuery round-trip and surfaces a typed error back to the agent’s retry loop.

(3) Empty-data guard. When SQL returns zero rows, the report writer is bypassed entirely; instead the system emits the executed SQL, a diagnostic explanation, and a suggestion to inspect filters. In early iterations we repeatedly observed the LLM fabricating findings “in the spirit of” the question when it had no data—a high-stakes failure mode in any production setting. The guard makes that mode unreachable.

4 Implementation and Data

4.1 Models and Infrastructure

ViSQL is implemented in Python (~3,600 LOC across 14 modules). Table 1 summarizes the model assignments. Local models use 4-bit quantization via *bitsandbytes*; serving is a thin *LlamaText*/*LlamaVision* wrapper over Hugging Face Transformers. Claude is called via the official Anthropic SDK. We use BigQuery as the data warehouse with three public datasets plus one synthesized A/B experiment.

4.2 Datasets and Schemas

We test on three industry verticals plus a synthetic A/B experiment. The datasets stress different aspects of the pipeline:

- **SEC quarterly financials** (Finance, 10 tables): XBRL long-format. Stresses join-planning—the planner must figure out how to pivot a key–value–style fact table.

Table 1. Model assignments. Local models are 4-bit quantized.

Stage	Model	Where	Role
Router	Llama-3.1-8B-Instruct	Local	Task class.
Schema link	MiniLM-L6-v2 + FAISS	Local	Retrieval
Vision (style)	Llama-3.2-11B-Vision	Local	StyleSpec
Planner	Claude Sonnet 4.5	API	CoT plan
SQL agent	Claude Sonnet 4.5	API	SQL gen
SQL baseline	SQLCoder-7B-2 + LoRA	Local	Spider eval
Report writer	Claude Sonnet 4.5	API	Narrative

- **theLook e-commerce** (Retail, 7 tables): the gold-standard demo dataset. Multi-table joins between orders, products, users, and inventory.
- **Google Analytics sample** (Web Analytics): nested **STRUCT** and repeated fields; sharded date-partitioned tables. Stresses BigQuery-specific dialect: **UNNEST**, wildcard-table queries, sub-selects on repeated records.
- **Synthetic A/B experiment**: variant assignment via a deterministic hash on user-id; treatment receives a synthetic +8% conversion lift. Used as the positive case for the A/B gate.

For Spider evaluation we use Spider 1.0 [18]: 7,000 train examples for both LoRA fine-tuning and the FAISS exemplar index, and a 200-example subset of the dev set for execution-accuracy benchmarking.

4.3 LoRA Fine-Tuning of SQLCoder-7B-2

We fine-tune SQLCoder-7B-2 [3] on Spider train using LoRA [6]: rank-16 adapters on the attention modules (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and the FFN modules (`gate_proj`, `up_proj`, `down_proj`), $\alpha=32$, dropout 0.05, learning rate 2×10^{-4} , two epochs, effective batch size 16 (batch 4, grad-accumulation 4). The base model is loaded in 4-bit (QLoRA-style) [4]. Total training time is roughly two hours on a single A100. The tuned adapter is used as a local baseline against the Claude-based agent. SQLCoder is not the production path—Claude is—but it serves as a reproducible local comparison and stress-tests our schema-linking and exemplar-retrieval stages independent of the API.

4.4 User Interface

The demo UI is a Streamlit + Flask application with seven tabs (Question, Route, Plan, SQL, Result, Chart, Report); the in-process backend shares the pipeline object so there is no IPC overhead. Reasoning traces, retry attempts, and the extracted **StyleSpec** are surfaced for inspection.

Table 2. Component-level evaluation results.

Suite	Metric	Value
Router	macro-F1 (60 labeled ex.)	0.92
Router (gate)	adversarial-probe accuracy	8/10
SQL on Spider	exec-acc (200-ex. dev slice)	0.74
Style imitation	mean ΔE_{76} (12 refs)	7.4
Style imitation	chart-type-hint accuracy	0.83
Reports (judge)	mean overall (6 q’s, 4 axes)	4.1/5
Reports	coverage of expected findings	0.78

5 Evaluation and Results

We evaluate each major component independently against a targeted suite, plus two qualitative end-to-end demonstrations of the A/B gate. Results are summarized in Table 2.

5.1 Router Accuracy

We hand-labeled 60 questions across the three datasets balanced across the five task classes, plus 10 *adversarial probes* specifically designed to test the A/B gate: questions that contain superficial “compare X vs Y” framing but are observational (e.g. “is conversion different between mobile and desktop”). The router achieves **0.92 macro-F1** on the labeled set. On the adversarial probes, the gate correctly demotes 8 out of 10—the two failures involve ambiguous phrasings that mention “test” without explicit experiment vocabulary, and we flag these as the natural next target for improvement.

5.2 SQL Execution Accuracy

We evaluate on a fixed 200-example slice of Spider 1.0 dev using the standard exec-accuracy metric (string-equivalent to the gold result set after canonicalization). Our Claude-backed SQL agent with FAISS-retrieved exemplars achieves **0.74 exec-accuracy**. As a local comparison, our LoRA-tuned SQLCoder-7B-2 with post-hoc Postgres-fixup heuristics also reaches roughly 0.74 on the same slice. The fine-tune improved the un-tuned SQLCoder base by approximately +13 percentage points, primarily on dialect compliance. We did not aim to beat the SOTA on Spider—the agent is engineered for BigQuery production use, not for benchmark gaming—but the parity result is a useful sanity check that retrieval-augmented Claude is competitive with a tuned specialist model on a public benchmark.

5.3 Style Imitation Self-Consistency

Evaluating style imitation against a human-judged ground truth is expensive. We instead use a *self-consistency* protocol: for each of 12 reference charts C_i , we extract a **StyleSpec** S_i , render a fresh chart C'_i using S_i on toy data, and re-extract a **StyleSpec** S'_i from C'_i . The metric

is the palette-averaged CIELAB ΔE_{76} between S_i and S'_i [15]. The intuition: if the rendering layer correctly applies the spec and the vision model is consistent, the round-trip distance should be small.

We obtain a mean $\Delta E_{76} = 7.4$ across the 12 references. The just-noticeable-difference (JND) threshold for color difference is generally cited at $\Delta E \approx 10$, so the round-trip is below the perceptible threshold on average. Chart-type-hint round-trip accuracy is 0.83.

5.4 Report Quality (LLM-as-Judge)

We hand-curated 6 reference questions spanning all five task classes and ran each end-to-end. The generated reports are scored by an independent Claude judge on four axes (factual grounding, completeness vs. expected findings, clarity, actionability) on a 1–5 Likert scale. The mean overall score is **4.1 out of 5**; coverage of expected findings is 0.78. The empty-data guard is verified across all 6 cases: in synthetic empty-result probes the report writer correctly refused to narrate and emitted the diagnostic.

5.5 End-to-End Demonstration: The A/B Test Gate

The architectural value of the strict A/B gate is best shown by two adjacent queries.

Real experiment (passes gate)

Q: “For the *checkout_redesign_2024* A/B test, did treatment lift conversion vs control?”

Route: **ab.test**.

Result: $\chi^2 = 412.7$, $p < .001$, lift = +8.1pp.

Report cites the statistics and the experiment by name.

Observational comparison (gated → single_chart)

Q: “Is conversion significantly different between mobile and desktop users?”

LLM proposed: **ab.test**. Gate demoted to: **single_chart**.

Rationale: no experiment-language in the question; no variant column in the schema.

Report describes the observed difference without causal language.

The second query is precisely the kind of question where an unconstrained pipeline would fabricate a randomized-trial framing. The gate makes that report unreachable by construction.

6 Discussion

What worked. The biggest single improvement to perceived report quality came from the empty-data guard. In an ungarded pipeline, the most damaging failure mode is not a wrong SQL query (which is at least debuggable from the SQL trace), but a confidently-narrated wrong answer when the SQL returned no rows. Hardcoding that failure mode away converted a class of silent failures into a loud, diagnosable one. The strict A/B gate operates on similar logic: most causal-claim mistakes are not LLM “reasoning errors,” they are routing errors. Fixing them at the routing layer is dramatically more reliable than asking the report writer to suppress them downstream.

Self-correction as a cheap insurance policy. The SQL agent’s retry loop (up to three attempts, with the BigQuery error message fed back into the next prompt) is the cheapest reliability win in the pipeline. Most first-attempt failures fall into a small number of recoverable categories that our error classifier distinguishes: **missing_object** (a hallucinated table or column), **syntax_error** (residual PostgreSQL idioms like `:: casts` or `ILIKE` that slipped past the dialect prompt), **wildcard_or_dataset_qualification** (BigQuery-specific qualification rules), and **type_error** (implicit-cast mismatches). For all of these, the model that just made the mistake usually fixes it on the next pass once the error string is in-context—the failure mode is local. The pre-execution table-existence check works in the same spirit but earlier in the pipeline: it converts a slow, expensive BigQuery round-trip failure into a fast, free local one. We do not present a separate retry-recovery metric—the Spider exec-accuracy of 0.74 already reflects the loop’s contribution, since retries fire transparently inside the agent. Instrumenting recovery rate explicitly is on the future-work list.

What surprised us. We expected LoRA-tuned SQL-Coder to substantially outperform Claude on Spider given that it is a specialist model trained on the exact distribution. The parity result (both around 0.74) is consistent with the broader recent observation that retrieval-augmented general-purpose LLMs are competitive with specialists on this benchmark when given good exemplars. We also did not expect the vision-style round-trip to land below the JND threshold; the structured-JSON intermediate appears to do most of the work, in line with our design slogan that “vision describes; matplotlib executes.”

Limitations.

- Spider 1.0 is increasingly easy for modern LLMs. BIRD [9] adds dirty data and ambiguous joins

and is a better proxy for production schemas; we did not evaluate there for resource reasons.

- The A/B gate is evaluated on a small adversarial set (10 probes). Multi-arm experiments, sequential testing, and ambiguous phrasings still need coverage. The 2 failures all involve “test” as a polysemous word.
- LLM-as-judge has known correlation issues with human raters on subjective axes. The 6-question report eval should be considered exploratory rather than confirmatory.
- The self-correction loop is not separately metrized; we report aggregate Spider exec-accuracy but not a retry-rate or recovery-rate breakdown. Without that, the marginal contribution of retries to the headline number is not separable from the model’s first-attempt skill.
- ViSQL currently assumes BigQuery; porting to Snowflake or Postgres would require swapping the dialect rules in the system prompt and the INFORMATION_SCHEMA introspection. The clean separation between schema management and the agent makes this straightforward but not trivial.

Future work. Four priorities. (i) Evaluate on BIRD-bench for a tougher production proxy. (ii) Expand A/B gate coverage with externally-labeled multi-arm and sequential-test probes. (iii) Instrument the SQL agent’s retry loop to report retry-rate, recovery-rate, and error-class distribution as first-class metrics. (iv) Promote the planner’s `<thinking>` trace from a UI feature to a first-class component, e.g. by re-feeding plan steps as constraints into a self-debugging SQL agent loop.

Conclusion. ViSQL demonstrates that a hybrid local/API pipeline with disciplined architectural guardrails can automate the analyst loop end-to-end without collapsing into the most common LLM failure modes. The combination of a strict A/B gate, live schema introspection, and an empty-data guard is, we believe, more important to the system’s usability than any single model choice.

Code and Data

The complete codebase, evaluation suites, and demo notebook are available in the project GitHub repository, along with the demo video. The Streamlit UI launches with a single function call after configuring API credentials.

References

- [1] Anthropic. 2025. Claude Sonnet 4.5: Model Card. (2025). Available at <https://www.anthropic.com/claude>.
- [2] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, et al. 2023. Binding Language Models in Symbolic Languages. (2023).
- [3] Defog AI. 2023. SQLCoder: A SOTA LLM for SQL Generation. Available at <https://github.com/defog-ai/sqlcoder>.
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).
- [5] Abhimanyu Dubey et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- [6] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. In *IEEE Transactions on Big Data*.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, et al. 2023. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [10] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2023).
- [11] Paula Maddigan and Teo Susnjak. 2023. Chat2VIS: Generating Data Visualizations via Natural Language using ChatGPT, Codex, and GPT-3 Large Language Models. *IEEE Access* 11 (2023), 45181–45193.
- [12] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).
- [13] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the Text-to-SQL Capabilities of Large Language Models. *arXiv preprint arXiv:2204.00498* (2022).
- [14] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [15] Gaurav Sharma, Wencheng Wu, and Edul N. Dalal. 2005. The CIEDE2000 Color-Difference Formula: Implementation Notes, Supplementary Test Data, and Mathematical Observations. *Color Research & Application* 30, 1, 21–30.
- [16] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Zhang, Zhao Yan, and Zhoujun Li. 2023. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. *arXiv preprint arXiv:2312.11242* (2023).
- [17] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- [18] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [19] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. 2023. LLM-as-a-Judge: A Survey. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).